

Tarea de Simulación: SIMULADOR DINÁMICO DE REDES DE COMPUTADORAS

Entorno de Desarrollo: Python (Pygame, NumPy/SciPy, Requests)

1. DESCRIPCIÓN GENERAL DEL PROYECTO

El objetivo de esta práctica es desarrollar un simulador visual e interactivo de una red de computadoras utilizando **Pygame y simpy**. El sistema deberá integrar tres componentes fundamentales de los métodos cuantitativos: **Teoría de Líneas de Espera (Colas)** para el modelado de tráfico y buffers, **Modelos de Inventario** para la gestión del almacenamiento de paquetes y el control de flujo, y el **Modelo de Asignación (Algoritmo Húngaro)** para la optimización y enrutamiento dinámico de cargas de trabajo entre enlaces o servidores de transmisión.

Adicionalmente, el software deberá registrar los eventos de desempeño, exportar las métricas de la simulación a un archivo plano de texto (.txt) y enviarlas automáticamente mediante peticiones HTTP a una API externa (por ejemplo, OpenAI API, Gemini API, Ollama o un backend REST propio) para obtener un diagnóstico, conclusiones y recomendaciones automatizadas de la eficiencia de la red.

2. REQUERIMIENTOS TÉCNICOS Y FUNCIONALES

Mapeo de Conceptos Matemáticos a la Red

Concepto Cuantitativo	Elemento en la Red	Modelado Matemático
Líneas de Espera	Llegada de Paquetes / Buffers de Routers	Proceso de Poisson (λ) y tiempo de servicio exponencial (μ). Métricas L, Lq, W, Wq.
Modelos de Inventario	Capacidad de Cola y Control de Flujo	Política de reabastecimiento s o Q*. Costos de almacenamiento (latencia) y penalización por

Concepto Cuantitativo	Elemento en la Red	Modelado Matemático
		ruptura (desbordamiento/pérdida de paquetes).
Modelo de Asignación	Balanceo Dinámico y Enrutamiento	Matriz de Costos C basada en latencia y saturación. Resolución óptima vía Algoritmo Húngaro.

A. Módulo de Teoría de Colas (Líneas de Espera)

- **Generación de Tráfico:** Las peticiones/paquetes deben generarse siguiendo un proceso de llegada de Poisson con tasa λ configurable por el usuario.
- **Procesamiento en Nodos:** Cada nodo transmisor (router/switch) actuará como un servidor con tasa de atención exponencial μ .
- **Cálculo de Métricas en Tiempo Real:** La simulación debe computar internamente:
 - L: Número promedio de paquetes en el sistema.
 - L_q : Número promedio de paquetes esperándolo en cola.
 - W: Tiempo medio de estancia total en la red.
 - W_q : Tiempo medio de espera en cola antes de ser transmitido.

B. Módulo de Gestión de Inventario (Buffer Control)

- **Capacidad del Buffer (S):** Cada router dispone de un tamaño máximo de almacenamiento en cola (capacidad de inventario).
- **Política de Reabastecimiento / Control de Flujo (s, Q):** Cuando el nivel del buffer disminuya por debajo del umbral mínimo s, el nodo enviará una señal de control de flujo para solicitar un nuevo lote de paquetes Q o solicitar la liberación del canal de entrada.
- **Costos de Sistema:**
 - *Costo de Mantener (Holding Cost):* Asociado al tiempo de permanencia de los paquetes en la memoria RAM / buffer.
 - *Costo de Ruptura / Penalización (Shortage Cost):* Ocurre ante un **Buffer Overflow** (pérdida de paquetes / *packet loss*). Cada paquete descartado suma una penalización económica/numérica al costo global de la simulación.

C. Módulo de Asignación Óptima (Algoritmo Húngaro)

- **Optimización de Enrutamiento:** En intervalos discretos de tiempo (Δt), el sistema

evaluará N flujos de datos pendientes de transmisión y M enlaces o nodos de salida disponibles.

- **Matriz de Costos (C_{ij}):** La matriz de asignación se calculará dinámicamente mediante la fórmula:
$$C_{ij} = \text{Latencia Actual del Enlace}_{ij} + \alpha \cdot \text{Saturación del Buffer del Nodo}_j$$
- **Resolución:** Se debe ejecutar el Algoritmo Húngaro (vía `scipy.optimize.linear_sum_assignment` o implementación propia) para determinar la asignación uno a uno que minimice el costo total de tránsito en la red.

D. Interfaz Gráfica con Pygame

- **Representación Visual:**
 - Nodos (Routers/Switches) representados mediante círculos cuyo color varía según la saturación de su buffer (Verde < 50%, Amarillo 50%-80%, Rojo > 80%).
 - Enlaces de comunicación representados mediante líneas dinámicas.
 - Paquetes de datos animados desplazándose en tiempo real a través de los enlaces.
- **Dashboard / Panel de Control:** Mostrar en pantalla el tiempo transcurrido, valor de λ y μ , tamaño actual de colas, tasa de paquetes perdidos y costo acumulado.
- **Interactividad:** Controles en teclado o pantalla para ajustar λ , simular la caída imprevista de un enlace o congelar la simulación.

E. Exportación de Datos e Integración con API externa

- **Exportación a TXT:** Al finalizar la simulación o presionar una tecla asignada (ej. E / ESC), el sistema guardará un resumen estructurado en un archivo llamado `reporte_simulacion.txt` con la siguiente estructura:

```
=====
                        REPORTE DE SIMULACIÓN DE RED
=====

Tiempo Total de Simulación: 120 s
Tasa de Llegada (lambda): 15.0 paquetes/s
Tasa de Servicio (mu): 18.0 paquetes/s
Capacidad de Buffer (S): 50 paquetes
Umbral Reabastecimiento (s): 10 paquetes

METRICAS OBTENIDAS:
- Paquetes Procesados: 1750
- Paquetes Perdidos (Overflow): 42
- Tasa de Pérdida: 2.40%
- Tiempo Medio en Cola (Wq): 0.12 s
- Promedio Paquetes en Sistema (L): 4.25
- Costo Total de Almacenamiento: $125.40
- Costo Total de Penalización (Ruptura): $420.00
```

- Costo Global del Sistema: \$545.40

=====

- **Envío e Integración con API (Análisis Automatizado):**

- El programa debe leer el contenido del archivo .txt o estructurar un payload JSON y realizar una petición HTTP POST a una API (ej. OpenAI, Gemini API, servidor Flask/FastAPI local).
- Prompt enviado a la API: *"Analiza los siguientes resultados de desempeño de un simulador de red basado en teoría de colas e inventario. Evalúa la tasa de pérdida de paquetes, tiempos de espera y costos, e indica conclusiones detalladas y 3 recomendaciones de optimización."*
- La respuesta recibida de la API debe ser mostrada en consola y guardada en la sección final del archivo reporte_simulacion.txt.

3. ENTREGABLES

1. **Código Fuente (.py):** Todo el script comentado de Pygame con la lógica de simulación, algoritmo húngaro, manejo de inventario y módulo HTTP.
2. **Archivo de Salida (.txt):** Muestra del reporte generado tras una corrida de prueba.
3. **Informe Técnico (.docx):** Explicación de los modelos matemáticos empleando las métricas obtenidas y capturas de la interfaz gráfica en ejecución.

Pautas.

Equipo máximo tres integrantes